

ENEL445 Sample Questions — Reproduced Questions and Worked Solutions

Note on completeness. The provided source PDF contains Questions 1–4 only. This .tex file fully reproduces and solves Q1–Q4, and includes placeholders for Q5–Q20 (not present in the source file).

Q1. Constrained Optimisation Basics

Problem. Consider the optimization problem

$$\min_{x_1, x_2} x_2^2 + x_1$$

subject to

$$x_1 \geq 1, \quad x_1 x_2 \geq 1.$$

(a) **Lagrangian.** Write the constraints in standard form:

$$g_1(x) = 1 - x_1 \leq 0, \quad g_2(x) = 1 - x_1 x_2 \leq 0.$$

With multipliers $\lambda_1, \lambda_2 \geq 0$, the Lagrangian is

$$\mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = x_2^2 + x_1 + \lambda_1(1 - x_1) + \lambda_2(1 - x_1 x_2).$$

(b) **Analytical solution.** Since $x_1 \geq 1 > 0$, the constraint $x_1 x_2 \geq 1$ implies $x_2 \geq 1/x_1$ (in particular $x_2 > 0$). Because x_2^2 is increasing for $x_2 \geq 0$, the best feasible choice for a given x_1 is

$$x_2^* = \frac{1}{x_1}.$$

Substitute into the objective to reduce to a single-variable problem:

$$\min_{x_1 \geq 1} \phi(x_1) := \frac{1}{x_1^2} + x_1.$$

Differentiate:

$$\phi'(x_1) = -\frac{2}{x_1^3} + 1.$$

Set $\phi'(x_1) = 0$:

$$-\frac{2}{x_1^3} + 1 = 0 \implies x_1^3 = 2 \implies x_1^* = 2^{1/3}.$$

This satisfies $x_1^* > 1$, so the bound $x_1 \geq 1$ is inactive at the optimum. Therefore

$$x_2^* = \frac{1}{x_1^*} = 2^{-1/3}.$$

(Optional KKT check.) At the optimum, $g_2(x) = 0$ so λ_2 may be positive; $g_1(x) < 0$ so $\lambda_1 = 0$.

- (c) **Problem type.** This is a (constrained) nonlinear optimization problem. Over the domain $x_1 \geq 1$ and $x_2 \geq 0$, it can be viewed as a *convex* optimization problem because:
- The objective $x_2^2 + x_1$ is convex.
 - The set $\{(x_1, x_2) : x_1 \geq 1, x_2 \geq 1/x_1\}$ is convex since $1/x_1$ is convex for $x_1 > 0$ and the constraint is its epigraph.
- (d) **Algorithm suggestion.** A standard choice is a **primal–dual interior-point (barrier) method**.
- **Key concepts.** Replace inequality constraints by barrier terms (e.g. $-\mu \log(x_1 - 1)$ and $-\mu \log(x_1 x_2 - 1)$), solve a sequence of unconstrained problems for decreasing $\mu \downarrow 0$, and use Newton steps on the KKT conditions (often in primal–dual form) to obtain fast local convergence.
 - **Why it is a good choice.** For convex problems, interior-point methods are reliable, typically converge in a modest number of Newton iterations, and directly exploit the structure of KKT conditions to find globally optimal solutions.

Q2. Duality

Problem. Consider the same problem as in Q1, except with the second constraint removed:

$$\min_{x_1, x_2} x_2^2 + x_1 \quad \text{subject to} \quad x_1 \geq 1.$$

- (a) **Convexity.** Yes, it is convex: the objective $x_2^2 + x_1$ is convex and the feasible set $x_1 \geq 1$ is a convex half-space.
- (b) **Lagrangian dual function.** Write $g(x) = 1 - x_1 \leq 0$ with multiplier $\lambda \geq 0$:

$$\mathcal{L}(x_1, x_2, \lambda) = x_2^2 + x_1 + \lambda(1 - x_1) = x_2^2 + (1 - \lambda)x_1 + \lambda.$$

The dual function is

$$q(\lambda) = \inf_{x_1, x_2} \mathcal{L}(x_1, x_2, \lambda).$$

Minimizing over x_2 gives $\inf_{x_2} x_2^2 = 0$ at $x_2 = 0$. For x_1 , the term is $(1 - \lambda)x_1 + \lambda$.

- If $\lambda \neq 1$, then $(1 - \lambda)x_1$ makes the infimum $-\infty$ (sending $x_1 \rightarrow \pm\infty$ in the appropriate direction).
- If $\lambda = 1$, the x_1 -term vanishes and $q(1) = 1$.

Hence

$$q(\lambda) = \begin{cases} 1, & \lambda = 1, \\ -\infty, & \lambda \geq 0, \lambda \neq 1. \end{cases}$$

The dual problem $\max_{\lambda \geq 0} q(\lambda)$ has optimal value $d^* = 1$ attained at $\lambda^* = 1$.

- (c) **Duality gap.** The primal optimum is attained at $x_2^* = 0$ and $x_1^* = 1$, giving

$$p^* = (0)^2 + 1 = 1.$$

Since $d^* = 1$, the duality gap is

$$p^* - d^* = 0.$$

Q3. Optimal Control

(a) **Model Predictive Control (MPC): principles and formulation.**

MPC repeatedly solves a finite-horizon constrained optimal control problem using the current measured (or estimated) state, applies only the first control input, then shifts the horizon forward and repeats (“receding horizon”).

A typical discrete-time formulation at time k is:

$$\min_{u_k, \dots, u_{k+N-1}} \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i}) + V_f(x_{k+N})$$

subject to dynamics and constraints

$$x_{k+i+1} = f(x_{k+i}, u_{k+i}), \quad x_{k+i} \in \mathcal{X}, \quad u_{k+i} \in \mathcal{U},$$

with x_k given. The applied control is $u_k = \kappa(x_k)$ where κ returns the first element of the optimizer.

Advantages (examples).

- Handles multivariable systems and hard constraints on states/inputs directly.
- Can incorporate nonlinear models and non-quadratic costs (when the solver supports it).

Disadvantages (examples).

- Requires solving an optimization problem online at each time step (computational cost).
- Performance depends on model quality; poor models can degrade closed-loop behavior.

(b) **Compute the MPC control law for the given integer system.**

Given:

$$x_{k+1} = x_k + u_k + 1, \quad x_k, u_k \in \mathbb{Z},$$

stage cost $c(x, u) = x^2 + u^2$, constraints

$$0 \leq x_k \leq 2, \quad -2 \leq u_k \leq 1,$$

terminal cost $V_f(x) = 2x$, and horizon $N = 2$.

For a given initial $x_k = x_0 \in \{0, 1, 2\}$, choose $(u_0, u_1) \in \{-2, -1, 0, 1\}^2$ to minimize

$$J = (x_0^2 + u_0^2) + (x_1^2 + u_1^2) + 2x_2$$

subject to

$$x_1 = x_0 + u_0 + 1 \in \{0, 1, 2\}, \quad x_2 = x_1 + u_1 + 1 \in \{0, 1, 2\}.$$

Enumerating feasible integer sequences yields the minimizing first control (the MPC feedback law $\mu(x_0)$):

- If $x_0 = 0$: optimal $(u_0, u_1) = (-1, -1)$, giving $(x_1, x_2) = (0, 0)$, and $\mu(0) = -1$.
- If $x_0 = 1$: optimal $(u_0, u_1) = (-2, -1)$, giving $(x_1, x_2) = (0, 0)$, and $\mu(1) = -2$.
- If $x_0 = 2$: optimal $(u_0, u_1) = (-2, -1)$, giving $(x_1, x_2) = (1, 1)$, and $\mu(2) = -2$.

Therefore, the MPC control law (for $x_k \in \{0, 1, 2\}$) is

$$u_k = \mu(x_k) = \begin{cases} -1, & x_k = 0, \\ -2, & x_k = 1, \\ -2, & x_k = 2. \end{cases}$$

(c) **Why add a terminal constraint/cost? Disadvantages?**

Why it helps.

- Encourages the finite-horizon optimizer to “aim toward” a desirable region (e.g. a neighborhood of an equilibrium), reducing myopic behavior.
- With appropriate design (terminal set and terminal cost), it can guarantee recursive feasibility and improve/ensure closed-loop stability.

Possible disadvantages.

- Can make the optimization more conservative or even infeasible if the terminal set is too small.
- Adds design effort (choosing a terminal set/cost) and may increase computation.

(d) **Bellman’s principle of optimality (in words).**

If a sequence of decisions is optimal from the initial state, then after taking the first decision and moving to the next state, the remaining decisions must form an optimal sequence for that new state. In other words, an optimal policy has “optimal substructure”: every tail of an optimal trajectory is itself optimal for its starting point.

Q4. Reinforcement Learning

(a) **When use value iteration / policy iteration vs. Q-learning?**

Use **value iteration** or **policy iteration** when you have a known Markov decision process model (transition probabilities and rewards) and a manageable state–action space, so you can perform dynamic programming sweeps.

Use **Q-learning** (or other model-free methods) when the model is unknown or too difficult to specify, and you learn from sampled interaction (online or from data), especially when you need incremental updates.

(b) **How policy iteration works (algorithm).**

Policy iteration alternates between:

- **Policy evaluation:** for a fixed policy π , compute its value function as the solution of the Bellman expectation equations

$$V^\pi(s) = \sum_{s'} P(s' | s, \pi(s)) \left(R(s, \pi(s), s') + \gamma V^\pi(s') \right),$$

solved exactly (linear system) or approximately (iterative evaluation).

- **Policy improvement:** update the policy greedily with respect to V^π :

$$\pi_{\text{new}}(s) \in \arg \max_a \sum_{s'} P(s' | s, a) \left(R(s, a, s') + \gamma V^\pi(s') \right).$$

Repeat until the policy no longer changes (it is optimal).

(c) **Why ε -greedy instead of 100% greedy in Q-learning?**

A purely greedy policy can get stuck exploiting early, noisy estimates of Q and fail to adequately explore actions that might be better. An ε -greedy policy injects exploration: with probability ε it tries a random action, improving state–action coverage and making it much more likely that the learned Q converges to the optimal action values (under standard assumptions such as sufficient exploration and suitable step sizes).

(d) **(Advanced) Make Q-learning on-policy; compare to SARSA.**

Standard Q-learning update (off-policy) uses the greedy next-action value:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

To make it **on-policy**, use the value of the *next action actually selected by the current behavior policy* (e.g. ε -greedy). Let $a' \sim \pi(\cdot | s')$. Then update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right].$$

This is exactly the **SARSA** update (State–Action–Reward–State–Action). Compared with Q-learning:

- SARSA is **on-policy**: it learns the action values for the same policy it follows (including exploration), often yielding more conservative behavior when exploration can lead to risky outcomes.
- Q-learning is **off-policy**: it learns values for the greedy policy while it may behave differently during learning; it can converge to the optimal greedy policy under suitable conditions but may behave more aggressively during learning.